

Mind the Gap - Find the Gap

Course: ME 2800 Design and Analysis of Cyber-physical Systems
Instructors: Dr. Cody Fleming
Date: December 15, 2025
Authors: Abhigyaan Deep, Micah Scheaffer, and Mukul Kulkarni

This file is best read as a Notion document: notion.so/Mind-the-Gap-Find-the-Gap.

Section 1

All our code exists under a modified fork, available at [tresabhi/f1tenth_gym_ros](https://github.com/tresabhi/f1tenth_gym_ros). The modified `/config/sim.yaml` we created to load Nuerburgring is as follows:

```
# MIT License

# Copyright (c) 2020 Hongrui Zheng

# Permission is hereby granted, free of charge, to any person
# obtaining a copy
# of this software and associated documentation files (the "S
# oftware"), to deal
# in the Software without restriction, including without limi
# tation the rights
# to use, copy, modify, merge, publish, distribute, sublicens
# e, and/or sell
# copies of the Software, and to permit persons to whom the S
# oftware is
# furnished to do so, subject to the following conditions:

# The above copyright notice and this permission notice shall
# be included in all
```

```
# copies or substantial portions of the Software.

# THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY K
IND, EXPRESS OR
# IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER
CHANTABILITY,
# FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
# AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAG
ES OR OTHER
# LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERW
ISE, ARISING FROM,
# OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTH
ER DEALINGS IN THE
# SOFTWARE.
```

```
bridge:
  ros__parameters:
    # topics and namespaces
    ego_namespace: "ego_racecar"
    ego_scan_topic: "scan"
    ego_odom_topic: "odom"
    ego_opp_odom_topic: "opp_odom"
    ego_drive_topic: "drive"
    opp_namespace: "opp_racecar"
    opp_scan_topic: "opp_scan"
    opp_odom_topic: "odom"
    opp_ego_odom_topic: "opp_odom"
    opp_drive_topic: "opp_drive"

    # transform related
    scan_distance_to_base_link: 0.0

    # laserscan parameters
    scan_fov: 4.7
    scan_beams: 1080
```

```

# map parameters
# map_path: "/home/exouser/flsim_ws/src/fltenth_gym_ros/maps/Montreal/Montreal_map"
map_path: "/home/exouser/flsim_ws/src/fltenth_gym_ros/maps/Nuerburgring/Nuerburgring_map"
map_img_ext: ".png"

# car parameters and scale
vehicle_params: "fltenth"
scale: 1.0

# opponent parameters
num_agent: 1

# ego starting pose on map
sx: 0.0
sy: 0.0
# stheta: -0.785398
stheta: 0.785398

# opp starting pose on map
sx1: 2.0
sy1: 0.5
stheta1: 0.0

# teleop
kb_teleop: True

```

And `follow_the_gap.py` received the following modifications (only `lidar_callback` received modifications with a few declared variables at the root of the class, before the `__init__`):

```

import numpy as np
import rclpy
from rclpy.node import Node
from ackermann_msgs.msg import AckermannDriveStamped
from sensor_msgs.msg import LaserScan

class FollowTheGap(Node):

    simulate_noise = False
    noise_mean = 0.0
    noise_std = 0.1

    smooth_noise = True
    smoothing_window_angle = np.radians(5)

    def __init__(self):
        super().__init__("follow_the_gap")

        self.get_logger().info("Follow the gap started.")
        self.drive_pub = self.create_publisher(AckermannDrive
Stamped, "/drive", 1)
        self.scan = self.create_subscription(
            LaserScan(), "/scan", self.lidar_callback, 1
        )
        self.scan
        self.lidar_fov = np.radians(180)
        self.max_speed = 5

        self.pub_drive(0.0, 0.0)

    def preprocess_lidar(self, ranges, window_size=15, max_val
ue=10, avoid=50):
        processed_data = ranges
        return processed_data

```

```

def find_max_gap(self, free_space_ranges):
    max_gap_start = 0
    max_gap_end = 0
    max_gap_length = 0

    return max_gap_start, max_gap_end

def lidar_callback(self, data):
    ranges = list(data.ranges)

    if self.simulate_noise:
        ranges = [
            max(
                r + np.random.normal(loc=self.noise_mean,
scale=self.noise_std), 0.0
            )
            for r in ranges
        ]

    if self.smooth_noise:
        window = int(len(ranges) * (self.smoothing_window
_angle / self.lidar_fov))
        smoothed = []

        for i in range(len(ranges)):
            start = max(0, i - window + 1)
            window_slice = ranges[start : i + 1]
            smoothed.append(sum(window_slice) / len(window_slice))

        ranges = smoothed

    numerator_s = 0
    denominator_s = 0

```

```

        for r in ranges:
            index = ranges.index(r)
            theta = self.lidar_fov * (index / len(ranges)) -
self.lidar_fov / 2

            if abs(theta) > np.radians(15):
                continue

            numerator_s += r
            denominator_s += 1

        speed = numerator_s / denominator_s
        speed = 1.7 * speed**0.65

        numerator_r = 0
        denominator_r = 0

        for r in ranges:
            index = ranges.index(r)
            theta = self.lidar_fov * (index / len(ranges)) -
self.lidar_fov / 2

            if abs(theta) > np.radians(50):
                continue

            weight_r = r**0.7
            numerator_r += weight_r * theta
            denominator_r += weight_r

        beta = 10.0 * 1.2 ** -(0.9 * speed)
        steering_angle = -beta * numerator_r / denominator_r

        self.pub_drive(speed, steering_angle)

def pub_drive(self, speed, steering_angle):
    drive_msg = AckermannDriveStamped()

```

```

        drive_msg.header.stamp = self.get_clock().now().to_ms
g()
        drive_msg.drive.speed = speed
        drive_msg.drive.steering_angle = steering_angle
        self.drive_pub.publish(drive_msg)
        self.get_logger().info(
            f"Control commands given, Speed: {speed}, Steering
g Angle: {steering_angle}."
        )

    def cleanup_on_shutdown(self):
        self.get_logger().info("Performing cleanup actions du
ring shutdown.")
        drive_msg = AckermannDriveStamped()
        drive_msg.header.stamp = self.get_clock().now().to_ms
g()
        drive_msg.drive.speed = 0.0
        drive_msg.drive.steering_angle = 0.0
        self.drive_pub.publish(drive_msg)

def main(args=None):
    rclpy.init(args=args)

    follow_the_gap = FollowTheGap()

    try:
        while rclpy.ok():
            rclpy.spin_once(follow_the_gap, timeout_sec=0.1)
    except KeyboardInterrupt:
        pass
    finally:
        follow_the_gap.get_logger().info("Ctrl+C received, st
opping car.")
        follow_the_gap.destroy_node()
        follow_the_gap.get_logger().info("Node shut down comp

```

```
letely.")
```

```
if __name__ == "__main__":  
    main()
```

Section 2

Brainstorming

First, to gain an understanding of the problem, we drew the map sections on the board and discussed ideas for an algorithm based on the car's actions. Most of our work consisted of trial and error with the ideas that we had. We started with a simple approach: follow the goal or furthest point. This worked well for mostly straight track, but broke on corners as it never prioritized moving away from walls.

We then had the car follow the gap for turning at a constant speed. The follow-the-gap algorithm introduced a bias to avoid nearby walls by biasing the steering angle away from the weighted average of all ranges, enabling the robot to steer clear of them, implemented as this equation:

$$\theta_{\text{target}} = -\beta \sum \frac{w_i \theta_i}{w_i}$$

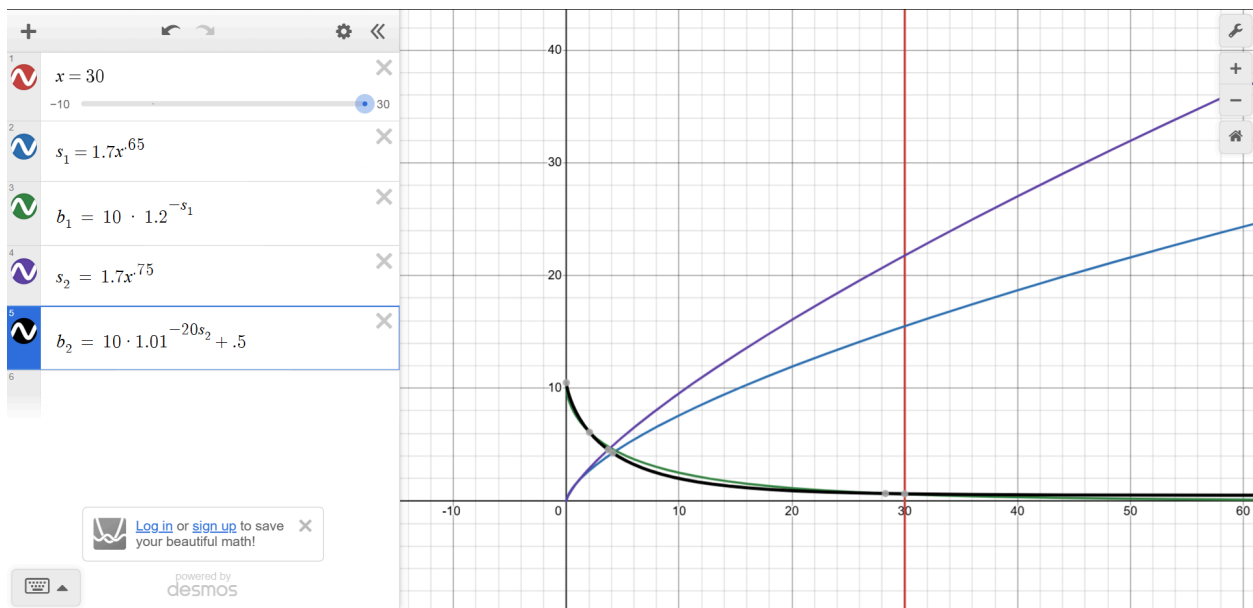
Here, $\beta = 1$ for the purposes of this initial iteration. Later, β will be a function of speed. w_i is the weight of the angle θ_i of individual ray i . We gave points further away less bias by setting $w_i = r_i^{-1}$

We then adjusted the FOV for this steering to an angle of about 120 degrees to ensure that what's behind the car does not bias the steering. Then we made an algorithm to set the speed proportional to the average distance the robot could see within a narrow FOV of 30 degrees in front of it, essentially allowing the bot to speed up given enough speed, dictated using the following equation:

$$s = \frac{1}{N} \sum r_i$$

We added a square root to the weight bias of steering in order to make the bias of the bias favor the close points. We then updated the steering FOV to 100 degrees, and we added a coefficient to the steering algorithm, which makes the steering more aggressive when the car is slower. This helps because the car automatically slows down when turns approach.

Below is the screenshot of some of the testing and validation we did for the equations. S1 and S2 is the speed equation that takes in the lidar range, and b1 and b2 are the steering coefficients that we use. Even though the purple curve has a higher top speed, it was unusable because it caused the car to crash. $x = 30$ is the max distance that the LIDAR could see, which gave us reference point for how to gauge how the curves behaved.



The plot above displays how we transform the speed s into the actual speed s' passed to the bot and decide on a dynamic β based on the speed.

Emulating and Dealing With Noise

Our model currently only deals with perfect LIDAR data; however, to handle noisy LIDAR data from a realistic system, we implemented a filter that smooths out the random spikes in distances. In this part of the code, we added a Gaussian noise with a mean of 0.0 and a standard deviation of 0.1 meters to every LiDAR reading. This simulates the small ± 10 cm jitter that real sensors produce. Next, we filtered this noise using a moving-average window that covers 5° of the LiDAR's field of view. The code converts that 5° into the right number of LiDAR indices, which is usually around 8 to 12 samples. We disable the noise simulation in real-life runs.

A video comparing the clean Lidar and the simulated noise runs (with smoothing) is available at <https://youtu.be/yF5h2EardRo>. A bag will be uploaded to Canvas of just the clean run on Nuerburgring.

Camera-Based Navigation

If the car had to navigate solely based on camera input, assuming it was a stereo camera, we could create an algorithm to estimate distance. Using this method, we would be able to build a similar algorithm using the stereo camera, which is functionally equivalent to the LiDAR we built.

This isn't similar to what YOLO did which primarily identified objects in the screen-space. We would need a more involved solution to create a better localized temporal world model than just screen-space coordinates.

Section 3

Surprisingly, our code worked well with the other tracks. Initially, we ran everything on the Nürburgring, but we also tested the code on the Catalunya, Sepang, and Austin Tracks. The most trouble we encountered was on the Austin track, specifically in the first turn, since it has a sharper turn than the other tracks. To remedy this, we simply made the steering more aggressive by decreasing the range of the LIDAR scans and incorporating steering as a function of speed (this is where we implemented β). As the speed slows down, the car is able to steer more

aggressively, allowing it to make tighter turns. The inclusion of β allowed the algorithm to work universally without breaking the laws of physics, to a reasonable extent.

The bag file for a Nuerburgring run will be uploaded to Canvas.

Bonus

The real-life track test was a disaster due to 2 main factors:

1. The real-life track was a mere 8m in length, compared to spans of 60m to 90m that are common on the simulation tracks.
2. The simulated physics was far different from the motion of the actual car. Jittery motions worked fine in simulation but decreased functionality on the actual car.

The reduced track length made the bot a little too cautious, making it give too little power to the motor. Friction ensure the care never moved forwards. After an hour of debugging, we scaled all range values by $30/8$ to mend this.

Furthermore, steering did not match the simulation 1 : 1; thus, we set $\beta = 1$ constant since it was being far too aggressive due to the low speeds. This produced an alright output.

We did have to manually back up quite often since the bot did not turn quick enough due to lag between the Lidar and response. The video for our best run is available at youtube.com/shorts/196fCJSk9I8.